

Lời giải HDU Multi-University Training Contest 9 năm 2021

Nguồn gốc: 2021 HDU Multi-University Training Contest 9

contest/hdu-2021-training-09-solutions.pdf

1 Problem A. NJU Emulator

Trước hết xét ý tưởng nhân đôi. Với mọi $N \geq 2$, ta có thể thu được N từ 1 bằng các thao tác $(\times 2)$ hoặc $(\times 2 + 1)$ trong $\lfloor \log_2 N \rfloor$ bước. Từ đó có một thuật toán dựng mọi N bằng $O(\log N)$ lệnh s86 như sau.

Dùng ba lệnh p1, dup, add 1 để đưa 1 và 2 vào ngăn xếp, rồi dùng p1 cùng các thao tác $(\times 2)$ hoặc $(\times 2 + 1)$ tổng cộng $\lfloor \log_2 N \rfloor$ lần để thu được N .

Cách này cần nhiều nhất $2\lfloor \log_2 N \rfloor + 5$ lệnh để dựng N , vẫn còn quá lớn. Để cải thiện, dùng phương pháp Four Russians: thay cơ số 2 bằng một cơ số lớn hơn k .

Với mọi $k \geq 2$ và $N \geq 2$, ta có thể thu được N bằng các thao tác $(\times k + p)$, $0 \leq p < k$, từ một giá trị x nào đó với $1 \leq x < k$, trong $\lfloor \log_k N \rfloor$ bước. Vì thế có thể dùng $2k - 1$ lệnh dạng p1, dup, add 1, dup, add 1, ... để đưa $1, 2, \dots, k$ vào ngăn xếp, sau đó dùng p1, add $(x-1)$ và $\lfloor \log_k N \rfloor$ thao tác dạng $(\times k + p)$ để thu được N .

Số lệnh khi đó nhiều nhất là $2\lfloor \log_k N \rfloor + 2k + 2$. Lấy $k = 7$ hoặc $8, 9, 10, 11, 12$, ta thu được thuật toán dựng mọi $N < 2^{64}$ trong không quá 60 lệnh s86, khá gần mục tiêu.

Để cải thiện thêm, chú ý rằng với một cơ số k , nếu trong ngăn xếp đã có $1, 2, \dots, \lfloor k/2 \rfloor$ và k , thì với mọi $x \leq k$ ta có thể lấy x trong 2 bước:

- dùng p1, add $x-1$ nếu $x \leq \lfloor k/2 \rfloor$;
- dùng dup, sub $k-x$ nếu $x > \lfloor k/2 \rfloor$.

Khi đó với $N > k$, ta có thể thu được N từ một x nào đó, $1 \leq x \leq k$, bằng các thao tác $(\times k + p)$ với $0 \leq p \leq \lfloor k/2 \rfloor$, hoặc $(\times k - p)$ với $1 \leq p \leq \lfloor k/2 \rfloor$, trong $\lfloor \log_k N \rfloor$ bước. Vì vậy chỉ cần $k + 1$ lệnh để đưa $1, 2, \dots, \lfloor k/2 \rfloor$ và k vào ngăn xếp, dựng x trong 2 bước, rồi áp dụng $\lfloor \log_k N \rfloor$ thao tác trên để thu được N .

Số lệnh nhiều nhất trở thành $2\lfloor \log_k N \rfloor + k + 4$. Lấy $k = 16$ hoặc $12, 14$, ta có thuật toán dựng mọi $N < 2^{64}$ trong không quá 50 lệnh s86.

2 Problem B. Just another board game

Trước hết xét chiến lược tối ưu nếu không cho phép thao tác “kết thúc ngay”. Ta có thể diễn giải lại thao tác của hai người chơi như sau.

Bắt đầu với $r = 1$ và $c = 1$. Ở mỗi lượt, nếu đến lượt Roundgod, anh ta có thể đặt c thành một giá trị bất kỳ thỏa $1 \leq c \leq m$. Nếu đến lượt kimoyami, anh ta có thể đặt r thành một giá trị bất kỳ thỏa $1 \leq r \leq n$. Sau k vòng, trò chơi kết thúc và giá trị trò chơi là $a_{r,c}$.

Xét quá trình theo chiều ngược. Nếu Roundgod đi ở lượt cuối, hiển nhiên anh ta nên chọn c là vị trí phần tử lớn nhất trên hàng hiện tại; do đó kimoyami, người đi áp chót, nên chọn hàng có giá trị lớn nhất theo hàng là nhỏ nhất. Tương tự, nếu kimoyami đi ở lượt cuối, anh ta nên chọn r là vị trí phần tử nhỏ nhất trên cột hiện tại; do đó Roundgod, người đi áp chót, nên chọn cột có giá trị nhỏ nhất theo cột là lớn nhất.

Còn một trường hợp đặc biệt $k = 1$, tức trò chơi kéo dài nhiều nhất một lượt. Khi đó rõ ràng Roundgod nên chọn phần tử lớn nhất trên hàng đầu tiên.

Bây giờ xét trường hợp cho phép thao tác “kết thúc ngay”. Khẳng định là thao tác này hầu như không thay đổi chiến lược của hai người, ngoại trừ nước đầu tiên của Roundgod.

Lý do là: nếu sau khi đối thủ vừa đi mà ta chọn thao tác thứ hai, kết quả không bao giờ tốt hơn việc không làm gì, tức giữ quân cờ đứng yên. Trong chiến lược tối ưu, đối thủ cũng sẽ không làm gì; hoặc nếu đối thủ cũng chọn thao tác thứ hai thì kết quả vẫn như vậy. Nếu không, đối thủ chọn một r hoặc c khác với nước vừa đi, điều này rõ ràng không tối ưu, và ta vẫn đạt được cùng giá trị mà không cần dùng thao tác thứ hai.

Tóm lại, đặt $rowmax_i$ là phần tử lớn nhất trên hàng thứ i , và $colmin_i$ là phần tử nhỏ nhất trên cột thứ i . Đáp án được tính như sau:

- nếu $k = 1$, đáp án là $rowmax_1$;

- nếu $k > 1$ và k lẻ, đáp án là

$$\max\left(a_{1,1}, \min_{i=1}^n rowmax_i\right);$$

- nếu k chẵn, đáp án là

$$\max\left(a_{1,1}, \max_{i=1}^m colmin_i\right).$$

Độ phức tạp thời gian tổng thể là $O(nm)$.

3 Problem C. Dota2 Pro Circuit

Trước hết xét cách tính $worst_i$ cho mỗi đội; $best_i$ có thể tính tương tự.

Với i và k cố định, để kiểm tra có tồn tại một phương án sao cho có k đội đạt điểm cao hơn đội i hay không, ta cho đội i nhận b_n điểm trong giải, và chọn k đội có điểm ban đầu cao nhất, trừ đội i , nhận lần lượt $b_1, b_2, \dots, b_k$ điểm trong giải. Ghép điểm ban đầu và điểm trong giải theo cách tham lam: đội có điểm ban đầu cao nhất nhận b_k điểm trong giải, đội có điểm ban đầu cao thứ hai nhận b_{k-1} điểm, và cứ như vậy. Sau đó kiểm tra xem mọi đội trong k đội được chọn có điểm cuối cùng lớn hơn hẳn đội i hay không. Phép kiểm tra này tốn $O(n)$ thời gian.

Nếu dùng tìm kiếm nhị phân để tính đáp án cho mọi i , độ phức tạp là $O(n^2 \log n)$, quá chậm. Quan sát đơn giản là nếu sắp xếp các đội theo điểm ban đầu, thì $worst_i$ không tăng. Vì vậy có thể dùng hai con trỏ để đạt tổng độ phức tạp $O(n^2)$.

4 Problem D. Into the Woods

Với một điểm $P(x, y)$, khoảng cách Manhattan từ P đến $O(0, 0)$ là

$$|x| + |y| = \max\{|x + y|, |x - y|\}.$$

Giả sử Roundgod đi qua đường đi $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Gọi A là khoảng cách Manhattan lớn nhất từ một điểm bất kỳ trên đường đi đến $(0, 0)$, khi đó

$$A = \max\left\{\max_{i=1}^n |x_i + y_i|, \max_{i=1}^n |x_i - y_i|\right\}.$$

Theo tính tuyến tính của kỳ vọng, giá trị cần tính là

$$E(A) = \sum_{k=1}^n \Pr[A \geq k] = n - \sum_{k=1}^n \Pr[A \leq k].$$

Một mẹo rất hữu ích và đẹp cho loại bước đi ngẫu nhiên hai chiều này là định nghĩa các biến ngẫu nhiên $a = x + y$ và $b = x - y$. Khi đó a và b độc lập và có cùng phân phối. Cụ thể, nếu vị trí hiện tại của Roundgod là (x, y) , thì ở mỗi bước, $x + y$ và $x - y$ độc lập tăng hoặc giảm 1 với xác suất $\frac{1}{2}$.

Bài toán được đưa về dạng sau. Ta đi ngẫu nhiên một chiều trên trục x trong n bước, bắt đầu từ $x = 0$; mỗi bước đặt ngẫu nhiên $x = x + 1$ hoặc $x = x - 1$ với xác suất $\frac{1}{2}$. Gọi p_k là xác suất toàn bộ đường đi nằm trong đoạn $[-k, k]$. Cần tính p_k với $k = 1, 2, \dots, n - 1$. Khi đó đáp án là

$$E(A) = n - \sum_{k=1}^{n-1} p_k^2.$$

Ta tiếp tục biến đổi sang một bài toán tương đương: xuất phát từ $(0, 0)$, đi n bước, mỗi bước chỉ được đi sang phải hoặc đi lên. Gọi c_k là số đường đi không chạm đường thẳng $y = x + (k + 1)$ và cũng không chạm đường thẳng $y = x - (k + 1)$. Khi đó

$$p_k = \frac{c_k}{2^n}.$$

Để tính số này, cần giải bài toán con sau: cho điểm P , đường thẳng $l_0 : y = x + a$ và $l_1 : y = x - b$, có bao nhiêu đường đi từ $(0, 0)$ đến P mà không vượt qua l_0 hoặc l_1 ? Có thể liên hệ với một bài toán đơn giản hơn, còn gọi là mở rộng của định lý bỏ phiếu Bertrand: cho điểm P và đường thẳng $l : y = x + a$, có bao nhiêu đường đi từ $(0, 0)$ đến P mà không vượt qua l ? Khi $a = 0$, số này chính là số Catalan.

Ý tưởng then chốt là nguyên lý phản xạ Andre. Theo nguyên lý phản xạ, khi chỉ có một đường giới hạn, đáp án là hiệu của hai hệ số nhị thức. Với phiên bản bao hàm - loại trừ của nguyên lý phản xạ, gọi $f(P)$ là ảnh phản xạ của P qua l_0 , $g(P)$ là ảnh phản xạ của P qua l_1 , và $C(P)$ là số đường đi từ $(0, 0)$ đến P , khi đó đáp án là

$$C(P) - C(f(P)) - C(g(P)) + C(g(f(P))) + C(f(g(P))) - \dots.$$

Suy ra có thể tính c_k bằng

$$c_k = \sum_{i=L}^R \binom{n}{i} + 2 \sum_{j \geq 1} (-1)^j \sum_{i=L}^R \binom{n}{i - (k + 1)j},$$

trong đó

$$L = \left\lceil \frac{n - k}{2} \right\rceil, \quad R = \left\lfloor \frac{n - k}{2} \right\rfloor.$$

Nếu tiền xử lý tổng tiền tố của $\binom{n}{i}$, thì c_k tính được trong $O(n/k)$ thời gian. Do đó độ phức tạp tổng thể là

$$\sum_{i=1}^n \frac{n}{i} = O(n \log n).$$

5 Problem E. Did I miss the lethal?

Bài này có thể diễn giải lại thành một trò chơi.

Ta có n lá bài trên tay; lá thứ i có hai giá trị tương ứng là d_i và a_i . Ta và đối thủ, tức người chia bài trong Hearthstone, luân phiên đi ở mỗi vòng:

1. Ta chọn một lá có d_i và a_i , bỏ lá đó và gây d_i sát thương.
2. Đối thủ chọn a_i lá và bỏ chúng.

Ta muốn tối đa hóa tổng sát thương gây ra, còn đối thủ muốn tối thiểu hóa nó. Cần tính đáp án khi cả hai chơi tối ưu.

Chú ý rằng mỗi a_i thuộc $\{1, 2, 3, 4\}$, nên ta chia toàn bộ n lá thành 4 đồng: mọi lá có $a_i = j$ được đặt vào đồng thứ j , với $1 \leq j \leq 4$. Gọi n_1, n_2, n_3, n_4 là số lá ban đầu trong từng đồng. Rõ ràng có một chiến lược tham lam: một khi ta hoặc đối thủ quyết định bỏ một lá khỏi một đồng, lá có sát thương lớn nhất trong đồng đó sẽ được chọn. Vì vậy với mỗi đồng, sắp xếp các lá theo d_i , rồi giải bằng quy hoạch động.

Đặt $dp[x_1][x_2][x_3][x_4][0]$ là đáp án khi đồng thứ j còn x_j lá với mọi $1 \leq j \leq 4$, và đến lượt ta chọn lá. Đặt $dp[x_1][x_2][x_3][x_4][k]$, với $k = 1, 2, 3, 4$, là đáp án khi đồng thứ j còn x_j lá và đến lượt đối thủ chọn k lá. Các chuyển trạng thái đều tính được trong $O(1)$.

Tổng số trạng thái là $5n_1n_2n_3n_4$, với

$$\sum_{i=1}^4 n_i \leq 200.$$

Số trạng thái đạt lớn nhất khi $n_1 = n_2 = n_3 = n_4 = 50$, tức $50^4 \cdot 5$, đủ nhỏ để dễ dàng qua giới hạn thời gian.

6 Problem F. Guess The Weight

Trước hết xét chiến lược tối ưu của uuzlovotree, phần này khá đơn giản: sau khi rút lá đầu tiên, anh ta nhìn các lá còn lại trong bộ bài rồi chọn “Small” hoặc “Large” tùy phía nào, so với lá đầu tiên đã rút, có nhiều lá hơn.

Quan sát then chốt là với mọi bộ bài, tồn tại một “điểm giữa” mp sao cho trong chiến lược tối ưu, uuzlovotree nên chọn “Large” nếu lá đầu tiên có số không vượt quá mp , và nên chọn “Small” nếu lá đầu tiên có số lớn hơn mp .

Giả sử đa tập lá bài hiện tại là SC , và số lá trong bộ là s , tức $|SC| = s$. Đặt

$$L = \{x \mid x \in SC, x \leq mp\}, \quad R = \{x \mid x \in SC, x > mp\}.$$

Khi xét mọi khả năng của hai lá đầu tiên trên bộ bài, xác suất rút được lá thứ hai là

$$\Pr[\text{DrawSecond}] = 1 - \frac{\sum_{x \in L, y \in L} [x \geq y] + \sum_{x \in R, y \in R} [x \leq y]}{s(s+1)}.$$

Do đó, nếu dùng cây đoạn để duy trì số lá và số cặp lá có giá trị khác nhau trong mỗi đoạn, ta có thể tính xác suất này trong $O(\log \max a_i)$. Điểm giữa cũng tìm được trong cùng độ phức tạp bằng cách đi xuống trên cây đoạn. Độ phức tạp thời gian tổng thể là

$$O(n + \max a_i + q \log \max a_i).$$

7 Problem G. Boring data structure problem

Duy trì phía trái, tức phần bên trái phần tử ở giữa, và phía phải, tức phần bên phải phần tử ở giữa, bằng hai danh sách liên kết đôi L và R . Khi đó hai thao tác chèn đầu tiên đều thực hiện được trong $O(1)$ thời gian cho mỗi truy vấn.

Sau khi thực hiện một thao tác, điều chỉnh kích thước hai danh sách liên kết đôi như sau:

- nếu $|L| > |R|$, chuyển phần tử cuối của L sang đầu R ;
- nếu $|L| < |R| + 1$, chuyển phần tử đầu của R sang cuối L .

Việc điều chỉnh này hiển nhiên có độ phức tạp khâu hao $O(1)$ cho mỗi truy vấn.

Khi xóa phần tử, chỉ cần đánh dấu phần tử đó là đã xóa và giảm kích thước danh sách liên kết đôi tương ứng đi một. Đây là thao tác “lười”; ta chỉ lan truyền việc xóa khi cần thực hiện thao tác trên phần tử đó, chẳng hạn di chuyển hoặc truy vấn. Vì thế thao tác xóa cũng có độ phức tạp khâu hao $O(1)$.

Do đó độ phức tạp thời gian tổng thể là $O(n + q)$.

8 Problem H. Integers Have Friends 2.0

Trước hết quan sát rằng nếu lấy $m = 2$ và chọn toàn bộ a_i lẻ hoặc toàn bộ a_i chẵn, ta luôn bảo đảm có đáp án ít nhất $\lceil n/2 \rceil$.

Chọn ngẫu nhiên hai chỉ số phân biệt $1 \leq x < y \leq n$. Xác suất để đáp án tối ưu chứa cả hai chỉ số này ít nhất là $\frac{1}{4}$. Nếu cả hai cùng nằm trong đáp án, thì m phải là một ước của $|a_x - a_y|$. Thực ra ta chỉ cần kiểm tra các thừa số nguyên tố của $|a_x - a_y|$. Mọi số dương x thỏa $x \leq 4 \cdot 10^{12}$ có nhiều nhất 11 thừa số nguyên tố phân biệt, và việc tìm tất cả chúng có thể làm dễ dàng trong $O(\sqrt{\max a_i})$ nếu tiền xử lý mọi số nguyên tố không vượt quá $2 \cdot 10^6$.

Lặp quá trình trên K lần. Xác suất sai nhiều nhất là $(3/4)^K$. Thời gian chạy là

$$O(K\sqrt{\max a_i} + 11Kn).$$

Lấy $K = 40$ cho xác suất lỗi không đáng kể và vẫn dễ dàng nằm trong giới hạn thời gian.

9 Problem I. Little Prince and the garden of roses

Rõ ràng có thể giải bài toán độc lập cho từng màu hoa hồng. Cố định một màu c . Ta cần gán màu cho cuống của các hoa hồng có màu c . Một phép quy về quen thuộc trên lưới như sau: xem lưới $n \times n$ là một đồ thị hai phía $G = (V = L \cup R, E)$ với $|L| = |R| = n$, và xem mỗi hoa hồng ở vị trí (u, v) là một cạnh nối đỉnh thứ u ở phía trái với đỉnh thứ v ở phía phải. Khi đó bài toán trở thành tô màu cạnh của đồ thị hai phía: gán màu cho mỗi cạnh sao cho không có hai cạnh kề cùng màu, đồng thời tối thiểu hóa số màu khác nhau được dùng.

Vậy cần bao nhiêu màu? Gọi Δ là bậc lớn nhất của một đỉnh trong đồ thị hai phía. Hiển nhiên cần ít nhất Δ màu. Ta có thể chứng minh rằng Δ màu cũng đủ với đồ thị hai phía; đây là trường hợp đặc biệt của định lý Vizing, phát biểu rằng chỉ số tô màu cạnh của đồ thị tổng quát là Δ hoặc $\Delta + 1$. Có hai cách làm.

Cách thứ nhất là một thuật toán xây dựng chạy trong $O(|E||V|)$. Tô màu các cạnh lần lượt theo một thứ tự nào đó. Gọi (x, y) là cạnh hiện tại. Nếu tồn tại màu c đang rảnh ở cả đỉnh x lẫn đỉnh y , ta tô trực tiếp cạnh (x, y) bằng c . Nếu không có màu như vậy, tồn tại một cặp màu c_1, c_2 sao cho c_1 có ở x nhưng không có ở y , còn c_2 có ở y nhưng không có ở x . Ta làm cho đỉnh y rảnh màu c_2 . Gọi z là đầu còn lại của cạnh kề y có màu c_2 . Nếu z rảnh màu c_1 , ta tô cạnh (x, y) bằng c_2 và đổi màu cạnh (y, z) sang c_1 . Như vậy ta thực hiện một phép hoán đổi theo dây xen kẽ. Nếu z không rảnh màu c_1 , gọi w là đầu còn lại của cạnh kề z có màu c_1 . Nếu w rảnh màu c_2 , lại có thể hoán đổi. Vì đồ thị là hai phía, cuối cùng ta sẽ tìm được một dây xen kẽ. Dùng DFS để tìm dây này; mỗi dây chứa không quá n đỉnh, nên độ phức tạp tổng thể là $O(|E||V|)$.

Cách thứ hai dễ hiểu hơn nhưng chậm hơn. Trước hết thêm các cạnh giả giữa các đỉnh trong L và R , có thể có bội cạnh, sao cho mọi đỉnh ở L và R đều có bậc Δ . Sau đó tìm một ghép hoàn hảo bất kỳ, tô

toàn bộ cạnh trong ghép bằng cùng một màu, xóa các cạnh của ghép, rồi lặp lại Δ lần. Cách này cũng cho một phép tô cạnh bằng Δ màu khác nhau. Tính đúng đắn có thể chứng minh bằng định lý Hall, ở đây lược bỏ. Độ phức tạp thời gian là

$$O(\Delta|E|\sqrt{|V|}),$$

và vẫn có thể được chấp nhận nếu cài đặt tốt.

Quy về theo n , độ phức tạp tổng thể là $O(n^3)$ cho cách thứ nhất và $O(n^{3.5})$ cho cách thứ hai.

10 Problem J. Unfair contest

Sau khi sắp xếp $a_1, a_2, \dots, a_{n-1}$ và $b_1, b_2, \dots, b_{n-1}$, điểm mới thêm a_n hoặc b_n có thể rơi vào ba loại: nằm trong s điểm cao nhất, nằm trong t điểm thấp nhất, hoặc không thuộc hai nhóm đó. Với mỗi loại, ta có một khoảng giá trị hợp lệ có thể có của a_n hoặc b_n ; phần còn lại chỉ là phân tích trường hợp cẩn thận và đầy đủ. Cần chú ý các trường hợp $s = 0$ hoặc $t = 0$.

Độ phức tạp thời gian là $O(n \log n)$ do cần sắp xếp.

11 Problem K. ZYB's kingdom

Trước hết xét cách giải khi $k = 1$ với mỗi truy vấn. Sự kiện loại 1 tương đương với việc: với mỗi cây con được tách ra bởi v , gọi s là tổng c trong cây con này; khi đó với mỗi đỉnh u trong cây con, ta cộng s rồi trừ c_u khỏi thu nhập của nó.

Tồn tại một thuật toán quen thuộc $O(q\sqrt{n} \log n)$ bằng cách tách các đỉnh có bậc không quá $\sqrt{n}$ và ít nhất $\sqrt{n}$, rồi dùng cây đoạn để cập nhật đáp án. Tuy nhiên có một cách khéo hơn nhiều, giải được trong $O(q \log n)$.

Ý tưởng là trước hết chọn gốc tùy ý cho cây rồi thực hiện phân rã nặng nhẹ. Với mỗi sự kiện loại 1, ta chỉ cập nhật cây con nối với cha của v và cây con nối với đứa con nặng duy nhất của v . Việc này tốn $O(\log n)$ thời gian, vì chỉ cập nhật hai cây con. Với mỗi sự kiện loại 2, sau khi truy vấn đáp án của v trên cây đoạn, phần nào còn thiếu? Ta cần cộng đóng góp của những cập nhật tại u sao cho u là tổ tiên của v , nhưng v không nằm trong cây con nối với con nặng của u . Nhờ tính chất của phân rã nặng nhẹ, chỉ có nhiều nhất $O(\log n)$ đỉnh như vậy. Vì thế, nếu lưu các cập nhật vào một mảng khi gặp sự kiện loại 1, và khi gặp sự kiện loại 2 thì lần lượt đi lên theo các chuỗi nặng, ta giải được mỗi sự kiện trong $O(\log n)$. Độ phức tạp tổng thể là $O(q \log n)$.

Khi $k > 1$ thì sao? Về cơ bản mọi thứ vẫn giống vậy; chỉ cần áp dụng cùng ý tưởng cho từng thành phần. Ở đây có thể cần xây một cấu trúc cây, còn gọi là cây ảo, theo k đỉnh bị cấm, rồi giải đệ quy trên cấu trúc cây này. Độ phức tạp tổng thể là

$$O\left(\sum k \log n\right).$$